

A refatoração de código do Dendê Eventos

Autores: Eli Kleber Carneiro Barreto
Elias Guideon Carneiro Barreto
Emerson Lucas Sacramento Lima
Kaio Viana Vilela Pinheiro

Agenda

O objetivo da apresentação é, com base na missão de refatorar o código, apresentar os tópicos seguintes.

Tópico 1: Introdução e Contexto

Contextualizar e explicar objetivos.

Tópico 2: Model.kt

Explicar arquivo que agrupa data classes e enums.

Tópico 3: Funções de componente e usos

Apresentar funções do arquivo Components.kt.

Tópico 4: Operações do repositório

Apresentar funções de Repositorio.kt.

Tópico 5: Funções principais refatoradas

Apresentar quatro funções refatoradas.

Tópico 6: Estrutura de arquivos final

Apresentar a estrutura de arquivos final do projeto.

Tópico 7: Resultado em Main.kt

Apresentação de resultado final do fluxo principal.

Tópico 8: Telas criadas

Apresentar telas criadas de funcionalidades.

'O que é Dendê Eventos?

- Aplicativo para divulgar, organizar e gerir eventos
- Da empresa baiana Dendê Softhouse (Salvador)

Sua linha do tempo

- Na primeira etapa, foi estruturada a primeira versão do aplicativo (código linear, zero funções e dados em memória)

DENDÊ EVENTOS

Um produto feito pela
DENDÊ SOFTHOUSE
para Você!

 Components.kt

 Evento.kt

 Ingresso.kt

 Main.kt

 Model.kt

 Repositorio.kt

 Usuario.kt

Segunda etapa de criação: Refatoração do código

- Criar arquivos para: data classes e enums, classes e funções para buscas, leituras e/ou escritas
- Adaptar main para criar/usar funções
- Organizar funções em arquivos para cada classe (usuário, evento e ingresso)

2 Criando Model.kt

- Selecionar data classes e enums da Main.kt e mover para Model.kt



Main.kt antes de mover (fun main roda em linha 54 do código)

```
54     fun main() {  
55  
56         // Configurando memória local  
57         val listaUsuarios = mutableListOf<Usuario>()  
58         val listaEventos = mutableListOf<Evento>()  
59         val listaIngressos = mutableListOf<Ingresso>()
```



Main.kt depois de mover (fun main roda em linha 1 do código)

```
1     fun main() {  
2  
3         // Configurando memória local  
4         val listaUsuarios = mutableListOf<Usuario>()  
5         val listaEventos = mutableListOf<Evento>()  
6         val listaIngressos = mutableListOf<Ingresso>()
```

Arquivo Model.kt concluído

```
data class DadosUsuario(
```

```
data class DadosEvento(
```

```
data class DadosIngresso(
```

```
enum class SexoUsuario { MASCULINO, FEMININO, NAO_INFORMADO }
```

```
enum class TipoUsuario { COMUM, ORGANIZADOR }
```

```
enum class TipoEvento {  
    SOCIAL, CORPORATIVO, ACADEMICO, CULTURAL_ENTRETENIMENTO, RELIGIOSO, ESPORTIVO,  
    FEIRA, CONGRESSO, OFICINA, CURSO, TREINAMENTO, AULA, SEMINARIO, PALESTRA, SHOW,  
    FESTIVAL, EXPOSICAO, RETIRO, CULTO, CELEBRACAO, CAMPEONATO, CORRIDA, OUTRO  
}
```

```
enum class ModalidadeEvento { PRESENCIAL, REMOTO, HIBRIDO }
```

3Arquivo Components.kt

- **Funções de leitura:** ReadInt, ReadDouble, ReadString
- **Utilidades:** definir intervalos, valores mínimos, tamanho mínimo de textos
- **Exemplos reais:**

```
readInt( print = "Opção: ", printErro = "Inválida", intervalo = 0 ≤ .. ≤ 3 )  
readDouble( print = "Preço: ", printErro = "Inválido", valorMinimo = 0.0 )  
readString( print = "Nome: ", printErro = "Nome curto", tamanhoMinimo = 2 )
```

readInt usada em menus com opções; readDouble usada em preços de ingressos (evento); readString usada em nomes (usuário e evento)

Arquivo Components.kt

- **Funções de leitura:** ReadInt, ReadDouble, ReadString
- **Utilidades:** definir intervalos, valores mínimos, tamanho mínimo de textos
- **Exemplos reais:**

```
readInt( print = "Ano: ", printErro = "Inválido", intervalo = 1920 ≤ .. ≤ 2020 )  
readDouble( print = "Taxa: ", printErro = "Inválida", valorMinimo = 0.0 )  
readString( print = "CNPJ: ", printErro = "Inválido", tamanhoMinimo = 2 )
```

readInt usada em ano em dataNascimento (usuário); readDouble usada em taxaEstorno (evento); readString usada em CNPJ (usuário)

Arquivo Components.kt

- **Função de escrita:** PrintTable
- **Utilidade:** imprimir tabelas bem formatadas
- **Exemplos reais:**

```
printTable("EVENTOS DISPONÍVEIS", colunas, linhas)
```

Função usada para imprimir uma tabela de eventos disponíveis em Feed de Eventos

```
printTable("SEUS EVENTOS:", colunas, linhas)
```

Função usada para imprimir uma tabela de eventos do usuário em Alterar Evento

4 Criando Repositorio.kt

- Selecionar listas criadas em Main.kt e mover para Repositorio.kt

Main.kt antes de mover (as listas são criadas antes do sistema)

```
1 fun main() {  
2  
3     // Configurando memória local  
4     val listaUsuarios = mutableListOf<Usuario>()  
5     val listaEventos = mutableListOf<Evento>()  
6     val listaIngressos = mutableListOf<Ingresso>()
```

Main.kt depois de mover (a fun main começa com o sistema)

```
1 fun main() {  
2     println("BEM-VINDO AO DENDÊ EVENTOS")
```

Operações do Repositorio.kt

- **.add**: Adicionar um valor a uma lista
- **.any**: Retornar qualquer valor da lista
- **.find**: Encontrar valor específico da lista
- **.count**: Realizar contagens de linhas
- **.filter**: Retornar uma lista filtrada
- **.forEach**: Gerenciar “para cada” linha
- **.maxByOrNull**: Seleciona o maior valor ou nada
 - **.sortedWith**: Ordenar lista

Funções do Repositorio.kt

Exemplos de criação/alteração

```
fun adicionarDadosUsuario(usuario: DadosUsuario) {  
    listaUsuarios.add(usuario)  
}
```

```
fun adicionarDadosIngresso(ingresso: DadosIngresso) {  
    listaIngressos.add(ingresso)  
}
```

```
fun adicionarDadosEvento(evento: DadosEvento) {  
    listaEventos.add(evento)  
}
```

.add

Refatoração de código (Dendê Eventos)

Funções do Repositorio.kt

Exemplos de busca/consulta

.find `fun buscarEventoDoIngresso` `fun buscarUsuarioCadastrado` **.find**

.any `fun buscarEventoPrincipalOrganizador` `fun buscarIngressoUsuario` **.find**

.find `fun buscarIdEventoOrganizador` `fun buscarProximoIngressoId` **.maxByOrNull**

.find `fun buscarIdEventoAtivo` `fun buscarProximoEventoId` **.maxByOrNull**

Funções do Repositorio.kt

Exemplos de listagem/ordenação

.filter

```
fun listarIngressosVendidos(idEvento: Int): List<DadosIngresso> {
```

```
fun listarIngressosReembolsados(idEvento: Int): List<DadosIngresso> {
```

.filter

```
fun listarIngressosUsuario(emailUsuario: String): List<DadosIngresso> {
```

.forEach, .find, sortedWith

```
fun ingressosUsuarioOrdenados(emailUsuario: String): List<DadosIngresso> {
```

.filter, .count, sortedWith

```
fun eventosDisponiveisOrdenados(dataHoje: LocalDateTime): List<DadosEvento> {
```

5 Funções a refatorar

- Assim como na primeira etapa, serão apresentadas:
 1. Alterar Perfil do Usuário
 2. Alterar Evento
 3. Listar eventos do organizador
 4. Listar ingressos

```
println("MENU PRINCIPAL - ÁREA LOGADA")
println("USUÁRIO: ${usuarioEncontrado.nome} (${usuarioEncontrado.email}).")
println("[1] Alterar Usuário [2] Visualizar Usuário [3] Desativar Usuário")

// Condicional para tornar menu dinâmico com base no tipo de usuário
when {
    usuarioEncontrado.tipoUsuario == TipoUsuario.COMUM -> {
        println("[4] Feed de Eventos [5] Visualizar Ingressos")
    }

    usuarioEncontrado.tipoUsuario == TipoUsuario.ORGANIZADOR -> {
        println("[6] Cadastrar Evento [7] Alterar Evento [8] Visualizar Eventos")
    }
}
```

Refatorando...

- Selecionar trechos de código de cada funcionalidade no Main.kt e mover para seu respectivo arquivo de classe.
- Substituir trechos (possíveis) por funções do repositório em cada uma.

 Evento.kt Ingresso.kt Usuario.kt

Como ficaram após refatorar?

 Components.kt

 Evento.kt

 Ingresso.kt

 Main.kt

 Model.kt

 Repositorio.kt

 Usuario.kt



Alterar Evento e Listar eventos do organizador



Listar ingressos



Alterar Perfil do Usuário

Refatoração de código (Dendê Eventos)

 Components.kt

 Evento.kt

 Ingresso.kt

 Main.kt

 Model.kt

 Repositorio.kt

 Usuario.kt

6 Estrutura de arquivos final

- Funcionalidades específicas movidas de Main.kt e criadas como funções nos arquivos:
 - Evento.kt, Ingresso.kt, Usuario.kt
- Funções que otimizam os formulários do sistema criadas em Components.kt
- Funções relacionadas a listas e dados em Repositorio.kt
- Data classes e enums em Model.kt

7 Resultado em Main.kt

- Somente loops de menus permanecem e fazem chamadas de funções.
- Aplicação de funções de componente

```
// Loop do menu inicial
do {
    println("MENU INICIAL")
    println("1. Cadastrar Usuário")
    println("2. Acessar Usuário")
    println("0. Sair")
    val opcaoMenuInicial = readInt()
```

```
when (opcaoMenuLogado) {
    1 -> {
        Usuario.alterarUsuario(usuarioEncontrado)
    }

    2 -> {
        Usuario.visualizarUsuario(usuarioEncontrado)
    }

    3 -> {
        Usuario.desativarUsuario(usuarioEncontrado)
        opcaoMenuLogado = 0
    }

    4 -> {
        Evento.feedEventos(usuarioEncontrado)
    }

    5 -> {
        Ingresso.visualizarIngressos(usuarioEncontrado)
    }
}
```

8Telas: Cadastrar evento



Elias Barreto - Organizador

NOVO EVENTO

Nome do Evento:

Página do Evento:

Descrição do Evento:

Elias Barreto - Organizador

DEFINIR PERÍODO DO EVENTO

DATA DE INICIO

Digite Somente Dia (DD):

Digite Somente Mês (MM):

Digite Somente Ano (AAAA):

Digite Somente Hora (HH):

Digite Somente Minuto (MM):

Elias Barreto - Organizador

DEFINIR PERÍODO DO EVENTO

DATA DE TERMINO

Digite Somente Dia (DD):

Digite Somente Mês (MM):

Digite Somente Ano (AAAA):

Digite Somente Hora (HH):

Digite Somente Minuto (MM):

Elias Barreto - Organizador

ATRIBUTOS DO EVENTO

Local do Evento:

Preço do Ingresso:

Aceita estorno de ingresso?

Se sim, qual a taxa de extorno %?

Elias Barreto - Organizador

ATRIBUTOS DO EVENTO

Local do Evento:

Preço do Ingresso:

Aceita estorno de ingresso?

Se sim, qual a taxa de extorno %?

Telas: Alterar evento

Both screens are for the 'ALTERAR EVENTO' (Edit Event) screen, showing the organizer 'Elias Barreto - Organizador'.

Left Screen: Seus Eventos:

Nº	Nome
1	Aniversário
2	Casamento
3	Formatura

Right Screen: Escolha uma opção:

Nº	Opção para Alterar
1	Nome
2	Página
3	Descrição
4	Período
5	Tipo
6	Evento Vinculado
7	Modalidade
8	Capacidade
9	Local
10	Preço/Estorno

Refatoração de código (Dendê Eventos)

Referências

- [OAT 1 - Descobrindo Kotlin - Google Docs](#)
- [OAT 2 - Refatorando o código - Google Docs](#)
- [ATIVIDADE DE SONDAGEM 4 - Google Docs](#)
- [Lista de Exercícios 5: Introdução a Funções - Google Docs](#)
- <https://kotlinlang.org/docs/collection-aggregate.html>
- <https://developer.android.com/codelabs/basic-android-kotlin-collections>